

UNIVERSIDADE ESTADUAL DO MARANHÃO  
PRÓ-REITORIA DE PESQUISA E PÓS-GRADUAÇÃO  
ESPECIALIZAÇÃO EM MICROELETRÔNICA PARA FRONT-END DIGITAL

**ANA PAULA RABELO DE FREITAS**  
**EDUARDO ASSUNÇÃO DE SOUSA**  
**HIRISLAYNE BATISTA RAMOS DOS SANTOS**  
**JEANE FERREIRA DE OLIVEIRA**  
**LYON RODRIGUES DE OLIVEIRA**  
**THIAGO LUAN FERREIRA DE SOUZA**

Implementação de uma Rede Modbus RTU Simulada em FPGA

**ANA PAULA RABELO DE FREITAS  
EDUARDO ASSUNÇÃO DE SOUSA  
HIRISLAYNE BATISTA RAMOS DOS SANTOS  
JEANE FERREIRA DE OLIVEIRA  
LYON RODRIGUES DE OLIVEIRA  
THIAGO LUAN FERREIRA DE SOUZA**

Implementação de uma Rede Modbus RTU Simulada em FPGA

Trabalho orientado apresentado à pós-graduação de nível *lato sensu* em microeletrônica para *front-end* digital da Universidade Estadual do Maranhão para obtenção de nota parcial da segunda fase do programa.

## **AGRADECIMENTOS**

À Associação para Promoção da Excelência do Software Brasileiro (Softex) por fomentar a capacitação de projetistas de sistemas digitais e pela parceria com o Ministério da Ciência, Tecnologia e Inovação (MCTI) para o Programa de Desenvolvimento de Competências em Sistemas Digitais.

À Fundação de Apoio ao Ensino, Pesquisa e Extensão (FAPEAD), que por meio do processo nº 177/2025, concedeu o apoio financeiro através da concessão da bolsa indispensável para a viabilização deste trabalho.

À Universidade Estadual do Maranhão (UEMA), instituição executora do projeto, por prover a infraestrutura necessária, o suporte administrativo e o corpo docente essencial para o desenvolvimento desta pesquisa.

## RESUMO

**Palavras-chave:**

## **ABSTRACT**

**Keywords:**

# SUMÁRIO

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>1 INTRODUÇÃO.....</b>                                                        | <b>8</b>  |
| 1.1 Problema Justificativa.....                                                 | 8         |
| 1.2 Motivação.....                                                              | 9         |
| 1.3 Hipótese de Pesquisa.....                                                   | 10        |
| 1.4 Objetivos.....                                                              | 11        |
| 1.4.1 Objetivo Geral.....                                                       | 11        |
| 1.4.2 Objetivos Específicos.....                                                | 11        |
| 1.5 Estrutura do Trabalho.....                                                  | 12        |
| <b>2 REFERENCIAL TEÓRICO.....</b>                                               | <b>13</b> |
| 2.1 Abordagens de implementação em FPGA.....                                    | 13        |
| 2.2 Modelo mestre-escravo.....                                                  | 16        |
| 2.3 Métodos de Verificação de Erros.....                                        | 16        |
| 2.3.1 Verificação de Quadro.....                                                | 17        |
| 2.3.1.1 Cálculo de Verificação de Erro (CRC-16).....                            | 17        |
| 2.4 Estrutura do quadro Modbus RTU.....                                         | 21        |
| 2.4.1 Função 03 - Read Holding Registers.....                                   | 21        |
| 2.4.2 Função 06 - Write Single Register.....                                    | 23        |
| 2.5 Respostas de exceção.....                                                   | 23        |
| <b>3 TRABALHOS RELACIONADOS.....</b>                                            | <b>25</b> |
| <b>4 METODOLOGIA DE PESQUISA.....</b>                                           | <b>28</b> |
| 4.1 Materiais e Métodos.....                                                    | 29        |
| 4.1.1 Módulo de Transporte Serial e Controle de Temporização.....               | 30        |
| 4.1.1 Módulo Mestre Modbus RTU.....                                             | 30        |
| 4.1.1 Módulo Escravo.....                                                       | 32        |
| 4.1.1 Módulo CRC-16.....                                                        | 32        |
| 4.1.2 Módulo de Registradores.....                                              | 33        |
| <b>5 RESULTADOS E AVALIAÇÃO.....</b>                                            | <b>34</b> |
| <b>6 CONCLUSÃO.....</b>                                                         | <b>35</b> |
| <b>Apêndice A - Módulo de Transporte Serial e Controle de Temporização.....</b> | <b>36</b> |
| <b>Apêndice B - Módulo Mestre.....</b>                                          | <b>36</b> |
| <b>Apêndice C - Módulo Escravo.....</b>                                         | <b>36</b> |
| <b>Apêndice X - Módulo CRC-16.....</b>                                          | <b>36</b> |
| <b>Apêndice Y - Módulo de Registradores.....</b>                                | <b>38</b> |
| <b>REFERÊNCIAS.....</b>                                                         | <b>40</b> |

## **LISTA DE ABREVIATURAS E SIGLAS**

# 1 INTRODUÇÃO

A comunicação industrial constitui um dos elementos fundamentais dos sistemas modernos de automação, pois permite a troca de informações entre sensores, atuadores, controladores lógicos programáveis, sistemas supervisórios e demais dispositivos presentes em ambientes produtivos. Em processos industriais, a confiabilidade da comunicação está diretamente relacionada à capacidade de monitorar variáveis, executar comandos, registrar eventos e garantir a operação coordenada de equipamentos distribuídos. Os protocolos de comunicação industrial assumem papel importante, pois definem regras padronizadas para o envio, recepção, interpretação e validação dos dados transmitidos entre os dispositivos (NIST, 2023).

Entre as soluções utilizadas em automação, o *Modbus* destaca-se por sua simplicidade, ampla adoção e documentação aberta. Desenvolvido para comunicação com controladores programáveis, o *Modbus* permanece presente em aplicações industriais devido à facilidade de implementação, ao baixo custo e à compatibilidade com diferentes meios físicos e arquiteturas de rede (MODBUS ORGANIZATION, 2012). Uma de suas variações mais utilizadas é o Modbus RTU (RTU significa Remote Terminal Unit), que emprega comunicação serial e estrutura compacta de mensagens, sendo comum em redes industriais baseadas em transmissão serial, sobretudo em aplicações sobre RS-485<sup>1</sup> (MODBUS ORGANIZATION, 2006).

## 1.1 Problema Justificativa

Em ambientes industriais de elevada exigência, a integridade operacional e o sincronismo na troca de dados entre elementos sensores, controladores e atuadores são requisitos críticos. Se, por um lado, o protocolo Modbus RTU é muito utilizado, se destacando como uma solução proeminente pela sua robustez e simplicidade; por outro, a sua execução lógica através de microcontroladores tradicionais, baseados em processamento sequencial via *software*, esbarra bastante em

---

<sup>1</sup> O RS-485 (ou EIA/TIA-485) é uma norma técnica que define as características elétricas de geradores e receptores para uso em sistemas de transmissão serial multiponto e balanceada. Operando por meio de comunicação diferencial sobre um par trançado de fios (geralmente acompanhado de um condutor comum de aterramento), o padrão oferece elevada imunidade a ruídos eletromagnéticos industriais e viabiliza a conexão física de múltiplos dispositivos em um mesmo barramento a distâncias de até 1.000 metros (MODBUS ORGANIZATION, 2006)



restrições de tempo real, originando falhas latentes e sobrecargas (*overhead*) de processamento que comprometem a confiabilidade do sistema (JOHNSON, 2020).

Essa vulnerabilidade dos sistemas baseados em *software* torna-se ainda mais crítica diante das restrições temporais rigorosas impostas pelo padrão Modbus, que exige intervalos de silêncio de 1,5 e 3,5 tempos de *caractere* para a correta delimitação e separação de quadros. Em sistemas embarcados de *software* que gerenciam múltiplas interrupções, garantir esse *timing* em altas taxas de transmissão (*baud rates*) pode sobrecarregar a unidade central de processamento (CPU) e gerar falhas de comunicação ou perda de pacotes (MODBUS ORGANIZATION, 2006).

Como alternativa para mitigar essas limitações, as FPGAs (*Field Programmable Gate Arrays*) surgem como uma solução por conta de seu paralelismo em nível de *hardware*, garantindo determinismo temporal absoluto e controle direto sobre os sinais de comunicação. No entanto, a literatura técnica demonstra que o desenvolvimento de controladores de comunicação sob essa lógica reconfigurável é um processo complexo (CHU, 2011). Projetos que não adotam uma separação clara entre a camada de transporte serial (UART) e a camada lógica do protocolo tendem a gerar códigos de *hardware* inflexíveis, difíceis de verificar e com baixa escalabilidade para a integração de múltiplos dispositivos em um mesmo barramento interno (CHU, 2011).

Diante desse cenário de transição entre as limitações do *software* e os desafios de projeto no *hardware* dedicado, emerge a questão central que este trabalho busca solucionar: “Como projetar e validar uma arquitetura digital modular e escalável em FPGA que garanta o cumprimento estrito das normas temporais e lógicas do protocolo Modbus RTU para múltiplos escravos, sem a dependência de processamento por *software*?”.

## 1.2 Motivação

A busca por respostas para a problemática apresentada é justificada pela necessidade de modernização tecnológica das redes industriais frente à transição para a Indústria 4.0 e à consolidação da Internet das Coisas Industrial (IIoT). Nesse novo panorama produtivo, são exigidos que equipamentos de chão de fábrica processem dados em velocidades cada vez maiores. Para atender a essa demanda,

as FPGAs vêm sendo bastante utilizadas em sistemas de automação e controle industrial, pois permitem a criação de arquiteturas *System-on-a-Chip* (SoC) com altíssimo desempenho e latência previsível (SZÁSZ, 2020).

A principal motivação para o desenvolvimento deste trabalho reside na necessidade de construir módulos de *hardware* (*IP cores*) dedicados que desonerem os processadores principais das tarefas repetitivas de comunicação. Ao transferir o controle do protocolo *Modbus RTU*, incluindo a verificação contínua do CRC-16 e o controle de temporização serial, todo para a lógica de portas programáveis, o sistema ganha um nível de determinismo que se mostra quase inatingível em implementações convencionais baseadas em processamento sequencial por *software* (HU et al., 2008).

Além da relevância tecnológica e de mercado para o ambiente de tecnologia operacional, este projeto é motivado pelo seu alto valor acadêmico e metodológico. A implementação e a simulação de uma rede completa, contendo um mestre e múltiplos escravos digitais multiplexados em um barramento interno, proporcionam a validação de uma arquitetura limpa em três camadas (Transporte, Protocolo e *Top-level*). Isso cria uma base sólida e reusável que poderá servir de referência para trabalhos futuros envolvendo a integração de sensores físicos, atuadores reais ou interfaces de comunicação padronizadas, como o RS-485 citado anteriormente.

### 1.3 Hipótese de Pesquisa

O presente trabalho baseia-se na premissa de que a implementação de uma rede Modbus RTU em nível de *hardware* sobre uma FPGA são capazes de superar as limitações de determinismo e latência observadas em implementações baseadas em *software* sequencial. A pesquisa parte do princípio que, ao transferir o controle de tarefas críticas, como o cálculo de redundância cíclica (CRC-16), a verificação de temporização e a interpretação de comandos, para blocos lógicos paralelos e dedicados, o sistema estará em conformidade com os requisitos normativos do protocolo sem gerar sobrecargas de processamento.

Para testar e validar essa hipótese, propõe-se a implementação de uma rede Modbus RTU simulada, composta por um módulo mestre e múltiplos escravos digitais operando concorrentemente. Cada escravo é dotado de um banco interno de

registradores, utilizado de maneira estratégica para emular o comportamento de sensores e atuadores industriais. O escopo de validação contempla a recepção e a transmissão de comandos, a execução de funções Modbus (especificamente FC03 e FC06), o acesso à memória, a geração autônoma de respostas, a verificação de erros por CRC-16 e a validação funcional.

A utilização de FPGA justifica-se pela possibilidade de descrever circuitos digitais em nível de *hardware*, permitindo a criação de arquiteturas paralelas, determinísticas e reconfiguráveis. Diferente de uma implementação em *software*, a FPGA permite representar blocos lógicos como máquinas de estados, registradores, unidades de controle, módulos de comunicação serial e circuitos de verificação de erro. Essa característica torna as FPGAs adequadas para aplicações que exigem paralelismo, controle temporal e implementação de arquiteturas digitais especializadas (BOUTROS; ARORA; BETZ, 2024).

Dessa forma, espera-se comprovar que o desenvolvimento de uma arquitetura estritamente modular e verificável contribui não apenas para o avanço prático do estudo do protocolo Modbus RTU, mas também para a consolidação de soluções em lógica programável voltadas para a comunicação industrial de alto desempenho.

## **1.4 Objetivos**

### **1.4.1 Objetivo Geral**

Desenvolver uma rede Modbus RTU simulada em linguagem *SystemVerilog*, composta por um módulo mestre e múltiplos escravos digitais, utilizando bancos de registradores internos para representar sensores e atuadores industriais simulados. O projeto visa contemplar a recepção e transmissão de comandos, interpretação de registradores, geração de respostas, detecção de erros e a validação funcional de todo o sistema por meio de simulação computacional.

### **1.4.2 Objetivos Específicos**

Para alcançar o objetivo geral proposto, os seguintes objetivos específicos foram traçados:

- Estruturar a arquitetura de uma rede Modbus RTU em três camadas modulares independentes: camada de transporte serial (UART), camada de protocolo Modbus e camada de integração *top-level*.
- Codificar o suporte às funções primárias do protocolo Modbus, com foco na Função 03 (*Read Holding Registers*) e na Função 06 (*Write Single Register*).
- Implementar a camada de segurança dos dados, desenvolvendo o módulo de cálculo de redundância cíclica (CRC-16) para validação da integridade dos frames transmitidos e tratamento de erros de comunicação.
- Validar o funcionamento do sistema por meio do desenvolvimento de *testbenches* e ambientes de simulação funcional, atestando o funcionamento da leitura/escrita em diferentes cenários (incluindo *timeouts* e endereços inexistentes) e extraíndo os dados de síntese da FPGA.

## 1.5 Estrutura do Trabalho

será escrito ao final do trabalho escrito

## 2 REFERENCIAL TEÓRICO

Os sistemas de automação industrial dependem da comunicação entre controladores, sensores, atuadores, instrumentos e sistemas supervisórios. Diante dessa dependência, os protocolos industriais estabelecem regras comuns para endereçamento, representação de dados, controle das transações e identificação de erros. Essa padronização permite que equipamentos com funções e fabricantes diferentes participem de uma mesma arquitetura de controle. Em ambientes de tecnologia operacional, a comunicação deve considerar disponibilidade, previsibilidade temporal e continuidade do processo, pois atrasos ou mensagens incorretas podem produzir efeitos sobre o sistema físico (Stouffer et al., 2023).

Para atender de maneira simples e eficiente essas exigências, destaca-se o protocolo Modbus, uma solução de arquitetura aberta consolidada no cenário de automação. Em sua camada de aplicação, o Modbus define uma unidade de dados composta pelo código de função e pelos parâmetros de dados associados. Essa unidade pode ser transportada sobre diferentes redes. Na linha serial, são acrescentados o endereço do servidor e o campo de verificação de erro, formando a unidade de dados de aplicação utilizada no modo RTU (Modbus Organization, 2006, 2012).

A documentação mais recente da camada de aplicação emprega os termos cliente e servidor, enquanto o guia de linha serial e grande parte da literatura utilizam mestre e escravo. No presente trabalho, o termo mestre designa o nó que inicia as transações, e o termo servidor ou escravo designa o nó endereçado que processa a solicitação. Essa diferença é somente terminológica e não representa dois mecanismos distintos para o escopo da rede serial estudada.

### 2.1 Abordagens de implementação em FPGA

A literatura mostra duas estratégias principais para implementar Modbus em FPGA. Na primeira, o protocolo é descrito diretamente em *Register-Transfer Level* (RTL) por máquinas de estados, registradores, temporizadores e blocos de CRC. Olmedo et al. (2011) abordam a camada de enlace Modbus em FPGA por VHDL, incluindo simulação e síntese. Pagare et al. (2020) também utilizam VHDL, UART e CRC em uma aplicação de controle. A principal vantagem dessa estratégia é tornar

explícitos o paralelismo e a temporização; em contrapartida, mudanças de protocolo exigem alterações no *hardware* descrito.

Na segunda estratégia, a FPGA incorpora um processador *soft-core* ou um sistema em chip programável e executa parte do protocolo em *software*. Chhatrawala, Jasani e Tilva (2016) utilizam *Nios II* em uma solução de aquisição de dados com Modbus, enquanto Urbina et al. (2018) integram Modbus e Profibus em uma plataforma SoC. Essa abordagem facilita a reutilização de bibliotecas e atualização funcional, mas introduz dependência de *software*, memória e tempo de execução do processador. Para este trabalho, a adoção de uma implementação RTL dedicada se mostra a estratégia mais adequada, uma vez que o objetivo do projeto exige a observação minuciosa da montagem do quadro, das restrições de temporização, do cálculo do CRC e da dinâmica da Máquina de Estados (FSM).

Para compreender o funcionamento dessa estrutura de controle no *hardware* reconfigurável, cabe definir que uma FSM representa o comportamento de um sistema por um conjunto limitado de estados, condições de transição e funções de saída. Em circuitos síncronos, o estado atual é armazenado em registradores e atualizado nas bordas do *clock*. A documentação de síntese da AMD caracteriza uma FSM por registrador de estado, função de próximo estado e função de saídas, e informa que ferramentas de FPGA reconhecem descrições nas formas *Moore* e *Mealy* (AMD, 2022).

A diferença entre esses dois modelos está na composição de suas saídas, em uma máquina de *Moore*, as saídas dependem principalmente do estado atual; enquanto em uma máquina de *Mealy*, também podem depender diretamente das entradas (CHU, 2011). Os protocolos frequentemente combinam as duas abordagens: sinais persistentes são associados aos estados, enquanto pulsos de aceitação ou transição respondem a *handshakes*. Independente da forma escolhida, as saídas devem possuir valores definidos em todos os caminhos combinacionais para evitar inferência não intencional de *latches*.

As FSMs são adequadas à implementação de protocolos de comunicação pelo fato de uma transação ocorrer como uma sequência ordenada de eventos: aceitação do comando, montagem da requisição, disponibilidade do transmissor, envio dos bytes, recepção da resposta, encerramento do quadro, validação e

conclusão. Representar cada etapa por um estado torna explícitas as condições de avanço e permite observar o comportamento em simulação.

Como exemplo prático, uma FSM de mestre pode ser decomposta em estados correspondentes à aceitação do comando, construção do quadro, transmissão, espera, validação e conclusão. Essa decomposição permite a separação entre o controle e o caminho de dados: a máquina decide quando cada operação ocorre, enquanto registradores, contadores e *buffers* armazenam os campos da transação. Para coordenar essa interação de forma síncrona e evitar gargalos, a incorporação de interfaces com confirmação de aceitação evitam a perda de *bytes* quando o transmissor permanece ocupado por mais de um ciclo.

A integridade do circuito digital exige ainda uma estratégia de inicialização e de prevenção de falhas contra estados órfãos. Para isso, o sinal de *reset* conduz o mestre ao estado inicial (*S\_IDLE*), limpa os contadores lógicos e invalida quaisquer respostas de transações anteriores. Ainda, um ramo *default* deve ser mapeado a fim de recuperar estados não reconhecidos. Ferramentas de síntese como o Vivado<sup>2</sup> também oferecem mecanismos de codificação e recuperação segura de FSM, embora esses recursos possam aumentar área ou reduzir desempenho (AMD, 2023). Para o escopo desse projeto, a recuperação explícita no RTL e a verificação de *reset* são suficientes para tornar o comportamento analisável e independente de uma ferramenta específica.

A confiabilidade operacional da rede simulada depende também de uma metodologia sistemática de teste. Em decorrência disso, a verificação da FSM deve abranger transições de estados sob condições normais e anormais de operação. Propriedades podem confirmar que comandos somente são aceitos no estado ocioso, que *bytes* não são perdidos durante a espera do transmissor, que uma resposta com CRC inválido nunca produz sucesso e que toda transação termina por resposta ou *timeout*. Para materializar e automatizar essa validação estrutural na FPGA, a linguagem SystemVerilog fornece construções para descrição RTL, *testbenches*, *assertions* e cobertura, favorecendo a rastreabilidade entre requisito e teste (IEEE, 2024).

---

<sup>2</sup> O AMD Vivado™ é um ambiente de desenvolvimento de *software* para FPGAs e SoCs adaptativos da AMD.

## 2.2 Modelo mestre-escravo

Para o cumprimento dessas propriedades temporais e lógicas, o Modbus adota uma disciplina de controle de acesso ao meio físico. No Modbus sobre linha serial, apenas um mestre controla as transações do barramento. O mestre constrói uma requisição, transmite o quadro e aguarda a resposta do escravo endereçado. Os escravos atuam de maneira reativa, de modo que não iniciam mensagens espontaneamente e não se comunicam diretamente entre si. Esse comportamento reduz a possibilidade de colisões no barramento simulado, desde que somente o nó cujo endereço corresponda ao quadro recebido habilite seu transmissor (Modbus Organization, 2006).

As transações padrão ocorrem de forma direcionada. Na prática, uma transação *unicast* é formada por uma requisição e, normalmente, por uma resposta. Se o escravo reconhecer a função e executar a operação, retorna uma resposta normal. Caso o quadro seja válido, mas a operação não possa ser executada, retorna uma resposta de exceção. Por outro lado, quando o endereço não existe, o quadro é destinado a outro nó ou a mensagem apresenta CRC inválido, não se espera uma resposta. Nesse cenário, o mestre precisa possuir um mecanismo de *timeout* para encerrar a tentativa e informar uma falha de comunicação.

Em contrapartida ao direcionamento individual de mensagens, o endereço zero é reservado às mensagens *broadcast*. Essas mensagens são destinadas aos servidores e não produzem resposta para evitar colisões no barramento. Como a primeira versão do projeto implementa apenas transações *unicast*, o mestre aceita endereços individuais e aguarda exatamente uma resposta. O suporte a broadcast pode ser acrescentado posteriormente, com um caminho específico que encerre a operação após a transmissão, mas está fora do escopo desta implementação.

## 2.3 Métodos de Verificação de Erros

A confiabilidade da comunicação na linha serial Modbus padrão fundamenta-se em dois níveis distintos de verificação de erros: a verificação de paridade (par ou ímpar), aplicada individualmente a cada caractere transmitido; e a verificação de quadro (LRC ou CRC), que atesta a integridade da mensagem em sua totalidade (MODBUS ORGANIZATION, 2006). Ambas as validações são



calculadas e anexadas pelo dispositivo emissor (seja ele mestre ou escravo) previamente à transmissão física, cabendo ao dispositivo receptor realizar o recálculo e a conferência estrutural durante o recebimento dos dados.

Para garantir o sincronismo e a tolerância a falhas na rede, o dispositivo mestre é parametrizado com um tempo limite de resposta (*timeout*). Este intervalo temporal é dimensionado para ser suficientemente longo, garantindo que qualquer escravo consiga processar a requisição unicast e formular a respectiva resposta. Caso o escravo detecte qualquer anomalia de transmissão, seja na paridade do caractere ou na assinatura do quadro, a norma técnica estabelece que a mensagem corrompida seja silenciosamente descartada. Consequentemente, o escravo omite a geração de uma resposta, forçando o esgotamento do tempo limite no mestre, o qual assume a responsabilidade pelo tratamento da exceção no nível da aplicação.

### 2.3.1 Cálculo de Verificação de Erro (CRC-16)

No modo de transmissão RTU, os quadros incorporam um campo de verificação de erros alicerçado no método de Verificação de Redundância Cíclica (CRC). Esse mecanismo atua como a principal ferramenta para garantir a integridade dos pacotes de dados transmitidos no barramento. O algoritmo do CRC-16 avalia toda a mensagem e sintetiza um valor numérico de 16 bits, o qual é implementado sob a forma de dois bytes independentes e anexado ao final do quadro antes da transmissão física (MODBUS ORGANIZATION, 2006)

Para a geração do CRC, o algoritmo oficial estabelece o uso do polinômio gerador constante 0xA001 (1010 0000 0000 0001). O procedimento de cálculo matemático segue um fluxo iterativo rigoroso, definido da seguinte forma:

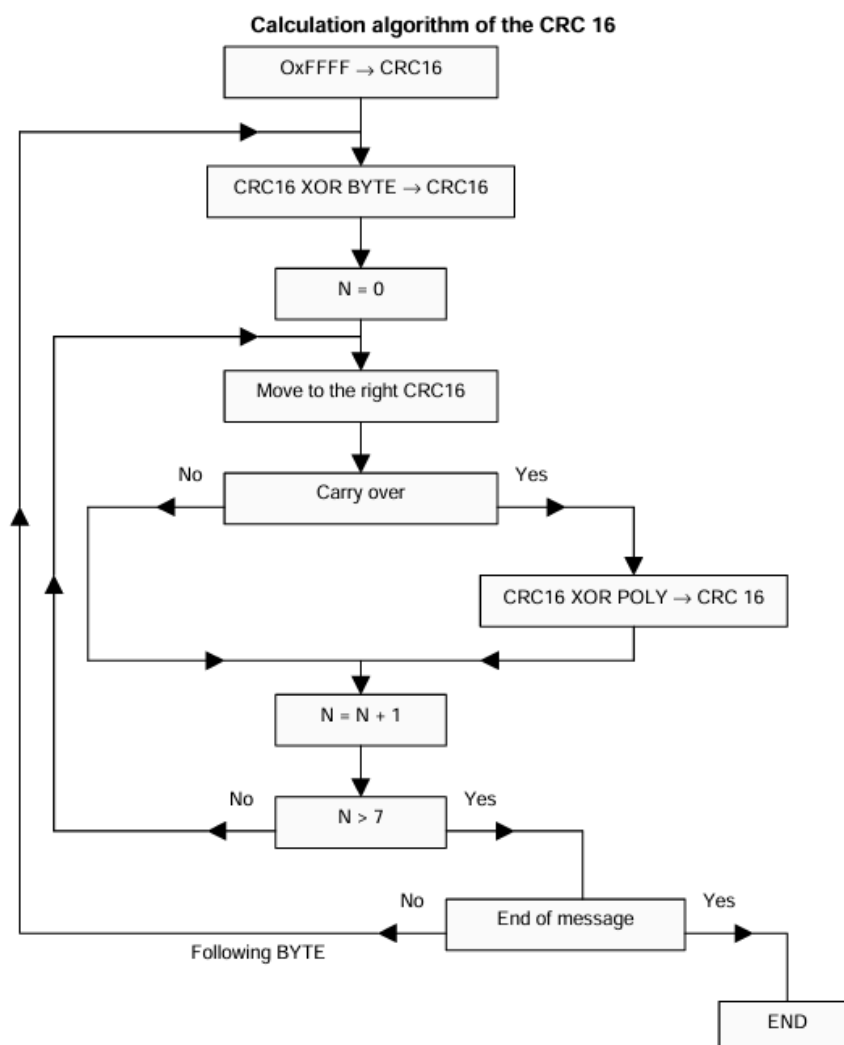
1. **Inicialização:** Um registrador de 16 bits é pré-carregado com o valor 0xFFFF (todos os bits em 1).
2. **Injeção do Dado:** Realiza-se a operação lógica XOR (OU Exclusivo) entre o primeiro byte (8 bits) da mensagem recebida e o byte de menor ordem (baixo) do registrador CRC atual, armazenando o resultado no próprio registrador.
3. **Deslocamento (*Shift*):** O registrador inteiro é deslocado um bit para a direita (em direção ao bit menos significativo - LSB), preenchendo o bit mais significativo com zero.

4. **Avaliação do Bit ejetado:** O bit que foi deslocado para fora (LSB) é extraído e avaliado.
  - Se o LSB for 0: Ocorre apenas um novo deslocamento para a direita.
  - Se o LSB for 1: O registrador sofre uma operação *XOR* com o valor do polinômio 0xA001.
5. **Iteração do Byte:** Os passos 3 e 4 são repetidos até que exatamente 8 deslocamentos tenham sido concluídos, o que indica que um byte completo foi processado.
6. **Iteração da Mensagem:** Repetem-se os passos de 2 a 5 para os próximos bytes da mensagem sequencialmente, até o término do pacote.
7. O valor final que permanece no registrador de 16 bits após o processamento de todos os caracteres corresponde ao valor calculado do CRC.

Por fim, no momento de acoplar o CRC calculado à mensagem para a transmissão física, o padrão normativo exige a inversão da ordem dos bytes (*Byte Swapping*). Assim, o byte de menor ordem (*Low-order byte*) deve ser transmitido primeiro no barramento serial, e em seguida o byte de maior ordem (*High-order byte*). Durante a etapa de recepção, o dispositivo de destino compara o valor do CRC recebido com a assinatura recalculada localmente. Se os dois valores não forem iguais, indica que ocorreu um erro de corrupção de dados durante o trajeto físico.

A **Figura X** ilustra o fluxograma de controle lógico exigido para a correta implementação do cálculo de redundância cíclica, detalhando as tomadas de decisão e os laços de repetição bit a bit.

**Figura X:** Fluxograma do algoritmo de cálculo do CRC-16.



Fonte: Modbus Organization (2006).

Para ilustrar a aplicação prática deste algoritmo, a **Figura Y** detalha o comportamento bit a bit dos registradores durante o cálculo de um quadro (*frame*) Modbus simples, composto apenas pelos caracteres 0x02 (Endereço do Escravo) e 0x07 (Código da Função). O algoritmo inicializa o registrador em 0xFFFF, realiza o *XOR* com o primeiro caractere (0x02) e executa os oito deslocamentos sucessivos. Na sequência, o processo iterativo é repetido para o segundo caractere (0x07).

**Figura Y:** Exemplo do cálculo passo a passo do CRC-16 para o quadro "02 07".

|                                  |        |      |      |      |        |
|----------------------------------|--------|------|------|------|--------|
| CRC register initialization      |        | 1111 | 1111 | 1111 | 1111   |
| XOR 1st character                |        | 0000 | 0000 | 0000 | 0010   |
| Flag to 1, XOR polynomial        | Move 1 | 1111 | 1111 | 1111 | 1101   |
|                                  |        | 0111 | 1111 | 1111 | 1110 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
| Flag to 1, XOR polynomial        | Move 2 | 1101 | 1111 | 1111 | 1111   |
|                                  |        | 0110 | 1111 | 1111 | 1111 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
|                                  | Move 3 | 1100 | 1111 | 1111 | 1110   |
|                                  |        | 0110 | 0111 | 1111 | 1111 0 |
|                                  |        | 0011 | 0011 | 1111 | 1111 1 |
|                                  | Move 4 | 1010 | 0000 | 0000 | 0001   |
|                                  |        | 1001 | 0011 | 1111 | 1110   |
|                                  |        | 0100 | 1001 | 1111 | 1111 0 |
|                                  | Move 5 | 0010 | 0100 | 1111 | 1111 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
|                                  |        | 1000 | 0100 | 1111 | 1110   |
|                                  | Move 6 | 0100 | 0010 | 0111 | 1111 0 |
|                                  |        | 0010 | 0001 | 0011 | 1111 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
|                                  | Move 7 | 1000 | 0001 | 0011 | 1110   |
|                                  |        | 0000 | 0000 | 0000 | 0111   |
|                                  |        | 0000 | 0000 | 0000 | 0111   |
| XOR 2nd character                |        | 0000 | 0000 | 0000 | 0111   |
|                                  | Move 1 | 1000 | 0001 | 0011 | 1001   |
|                                  |        | 0100 | 0000 | 1001 | 1100 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
|                                  | Move 2 | 1110 | 0000 | 1001 | 1101   |
|                                  |        | 0111 | 0000 | 0100 | 1110 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
|                                  | Move 3 | 1101 | 0000 | 0100 | 1111   |
|                                  |        | 0110 | 1000 | 0010 | 0111 1 |
|                                  |        | 1010 | 0000 | 0000 | 0001   |
|                                  | Move 4 | 1100 | 1000 | 0010 | 0110   |
|                                  |        | 0110 | 0100 | 0001 | 0011 0 |
|                                  |        | 0011 | 0010 | 0000 | 1001 1 |
|                                  | Move 5 | 1010 | 0000 | 0000 | 0001   |
|                                  |        | 1001 | 0010 | 0000 | 1000   |
|                                  |        | 0100 | 1001 | 0000 | 0100 0 |
|                                  | Move 6 | 0010 | 0100 | 1000 | 0010 0 |
|                                  |        | 0001 | 0010 | 0100 | 0001 0 |
|                                  |        | 0001 | 0010 | 0100 | 0001 0 |
| The CRC 16 of the frame is then: |        | 4112 |      |      |        |

Fonte: Modbus Organization (2006, p. 41).

Após a conclusão de todas as operações lógicas com o polinômio gerador 0xA001, o valor final obtido internamente no registrador de 16 bits é 0x1241. Atendendo à regra de transmissão do protocolo para o campo de verificação de erro, a ordem dos bytes deve ser invertida no momento do envio para o barramento serial. Dessa forma, o byte menos significativo (no exemplo, 0x41) é transmitido imediatamente após o pacote de dados, sendo finalizado pelo byte de ordem superior (no exemplo, 0x12).

## 2.4 Estrutura do quadro Modbus RTU

O quadro de dados do Modbus RTU é estruturado por um byte de endereço, um byte de código de função, um campo de dados de tamanho variável e dois bytes reservados para o cálculo de verificação de erro (CRC-16). De acordo com a norma oficial, o tamanho máximo do pacote completo, denominado Unidade de Dados da Aplicação (ADU), é de 256 bytes (MODBUS ORGANIZATION, 2006, 2012).

Diferentemente do modo ASCII, o modo RTU não emprega caracteres explícitos para a marcação de início e fim de quadro. A delimitação das mensagens ocorre exclusivamente por meio de temporização, utilizando intervalos de silêncio no barramento serial. Um período de silêncio contínuo igual ou superior a 3,5 tempos de caractere ( $t_{3.5}$ ) identifica a fronteira que separa quadros distintos, indicando o término de uma mensagem e o possível início de outra.

Paralelamente, o protocolo exige um rigoroso sincronismo durante a transmissão dos dados. Caso ocorra uma pausa de comunicação superior a 1,5 tempo de caractere ( $t_{1.5}$ ) entre dois bytes consecutivos no interior de uma mesma mensagem, o pacote em trânsito deve ser considerado corrompido, resultando na invalidação sumária do quadro por parte do dispositivo receptor.

### 2.4.1 Função 03 - Read Holding Registers

A Função 03, denominada *Read Holding Registers*, é responsável por solicitar a leitura de uma sequência contígua de registradores de retenção (*holding registers*). Esses registradores possuem 16 bits de largura e armazenam parâmetros de configuração, valores intermediários ou variáveis de processo cuja interpretação semântica é definida pela aplicação (MODBUS ORGANIZATION, 2012). O protocolo Modbus não associa automaticamente unidades de engenharia, sinais ou tipos numéricos ao conteúdo transmitido; consequentemente, a mesma palavra de 16 bits pode representar um número inteiro, parte de um valor de 32 bits ou um conjunto de *flags*, dependendo exclusivamente do mapa de memória do equipamento.

Na Unidade de Dados da Aplicação (ADU) da requisição, o campo de endereço inicial possui dois bytes e identifica o primeiro registrador a ser lido. O campo subsequente também possui dois bytes e informa a quantidade de registradores solicitados. A especificação oficial estabelece que uma quantidade

válida deve estar na faixa de 1 a 125 registradores por requisição (MODBUS ORGANIZATION, 2012). Esse limite decorre do tamanho máximo estipulado para a ADU e da necessidade de alocar, na resposta, dois bytes para cada registrador lido. Na linha serial, o quadro completo da requisição ocupa oito bytes: endereço do servidor (escravo), código da função (0x03), endereço inicial alto, endereço inicial baixo, quantidade alta, quantidade baixa, CRC baixo e CRC alto (MODBUS ORGANIZATION, 2006, 2012).

A resposta normal a esta solicitação mostra o endereço do servidor, o código 0x03, a contagem de bytes (*byte count*), o conteúdo lido dos registradores e o CRC. Como cada registrador ocupa dois bytes, a contagem de bytes será sempre um número par, correspondendo ao dobro da quantidade solicitada. Em cada registrador, o byte mais significativo (*High-order byte*) é transmitido antes do menos significativo (*Low-order byte*). O dispositivo mestre deve verificar o endereço, a função, o tamanho e o CRC antes de disponibilizar os valores à camada de aplicação.

Por exemplo, uma solicitação ao escravo de endereço 0x01 para a leitura de dois registradores a partir do endereço 0x0010 possui, antes da inserção do CRC, a seguinte sequência hexadecimal: 01 03 00 10 00 02. Uma resposta hipotética retornando os valores 0x1234 e 0x5678 apresentará a estrutura 01 03 04 12 34 56 78, finalizada pelo CRC correspondente a esses sete bytes. Caso a quantidade solicitada seja zero, exceda o limite de 125, ou ultrapasse o mapa de memória disponível no escravo, o servidor deve gerar um código de exceção, abstendo-se de retornar dados parciais.

No contexto de projeto de hardware, a recepção e o armazenamento de múltiplos registradores podem ser organizados por meio de *buffers*, filas FIFO, memória compartilhada ou interfaces de fluxo. A escolha arquitetural depende do volume máximo de dados, do paralelismo requerido e do método de consumo das variáveis pela aplicação, sem alterar as normativas de formato definidas pelo protocolo.

#### 2.4.2 Função 06 - Write Single Register

A Função 06, designada como *Write Single Register*, possui a finalidade de modificar o conteúdo de um único registrador de retenção (*holding register*). A Unidade de Dados da Aplicação (ADU) da requisição é composta pelo endereço do registrador alvo e pelo novo dado a ser escrito, ambos com 16 bits de largura. No modo de transmissão RTU, o quadro correspondente possui oito bytes de comprimento estruturados na seguinte ordem: endereço do servidor (escravo), código da função (0x06), endereço alto, endereço baixo, valor alto, valor baixo, CRC baixo e CRC alto. O valor transmitido pode abranger toda a extensão da faixa de 0x0000 a 0xFFFF, embora sua validade semântica esteja sujeita a restrições impostas pela camada de aplicação do dispositivo receptor (MODBUS ORGANIZATION, 2012).

Ao executar a escrita com sucesso, o servidor responde repetindo integralmente o código da função, o endereço e o valor recebidos. Essa resposta em forma de eco não constitui uma nova leitura do registrador, mas confirma que a solicitação foi reconhecida e processada. Por isso, o mestre não deve validar apenas função e CRC: também precisa comparar os quatro bytes de endereço e valor com os campos armazenados no início da transação.

Considere, por exemplo, a escrita do valor 0xABCD no registrador 0x0002 do servidor 0x01. Os bytes anteriores ao CRC são 01 06 00 02 AB CD. A resposta normal repete a mesma sequência e acrescenta um CRC próprio. Se o servidor devolver endereço ou valor diferente, mesmo com CRC correto, a mensagem não corresponde à transação pendente e deve ser classificada como resposta inválida.

Em situações de anomalia, a tentativa de escrita em um registrador inexistente deve desencadear a geração da exceção 02 (Endereço de Dados Ilegal), ao passo que a recepção de um valor incompatível com as regras de negócio da aplicação resulta na exceção 03 (Valor de Dados Ilegal). Reitera-se, portanto, que a transação somente é considerada concluída com sucesso após a validação exaustiva do quadro de resposta. No escopo da Função 06, este procedimento engloba a verificação do comprimento do pacote, do endereço da unidade, do código da função, da integridade do CRC e da correspondência exata dos campos de dados em relação à solicitação matriz.

## 2.5 Respostas de exceção

Uma resposta de exceção é uma mensagem Modbus válida utilizada quando o servidor recebe uma solicitação íntegra e endereçada corretamente, mas não consegue executá-la. O código da função solicitada é retornado com o bit mais significativo ativado. Desse modo, FC03 torna-se 0x83 e FC06 torna-se 0x86. O byte seguinte contém o código de exceção, e o quadro RTU é encerrado pelos dois bytes do CRC (Modbus Organization, 2012).

A exceção 01, *Illegal Function*, informa que o código de função não é reconhecido ou não é suportado pelo servidor. A exceção 02, *Illegal Data Address*, indica que o endereço, ou a combinação entre endereço inicial e quantidade, não corresponde a objetos disponíveis. A exceção 03, *Illegal Data Value*, sinaliza que o valor presente na requisição está fora do domínio permitido; na FC03, quantidade igual a zero ou superior ao limite é um exemplo. A exceção 04, *\*Server Device Failure\**, pode indicar erro irreversível durante a tentativa de executar a operação (Modbus Organization, 2012).

O mestre deve primeiro confirmar endereço, comprimento e CRC para somente depois interpretar o bit de exceção. Uma mensagem com código 0x83, mas CRC incorreto, não é uma exceção confiável: trata-se de um quadro corrompido. Quando a exceção é válida, convém disponibilizar separadamente o estado da transação e o código retornado, permitindo que a aplicação diferencie uma função não suportada de um endereço inexistente.

É importante distinguir a exceção de falha de comunicação: a exceção comprova que houve resposta do servidor, embora a operação tenha falhado; as falhas de comunicação ou respostas inválidas são CRC incorreto, endereço inesperado, função incompatível, quadro incompleto e *timeout*. Essa distinção contribui para o diagnóstico da comunicação, pois permite ao mestre diferenciar falhas transitórias de transmissão de erros associados à própria solicitação. Em situações de *timeout*, por exemplo, uma nova tentativa pode ser adequada, já que a ausência de resposta pode decorrer de ruído, atraso ou indisponibilidade momentânea do escravo. Em contrapartida, quando o servidor retorna uma exceção por endereço ilegal, a repetição indefinida da mesma requisição tende a reproduzir o erro, pois a causa está nos parâmetros enviados e não no meio de comunicação.



### 3 TRABALHOS RELACIONADOS

No cenário atual da pesquisa sobre implementação do protocolo Modbus RTU em FPGA, diversos autores propuseram soluções voltadas ao controle de dispositivos industriais, aquisição de dados e conversão entre protocolos. O trabalho de McBride (2023) desenvolveu uma abordagem baseada em núcleos de propriedade intelectual (IP cores) em VHDL para controlar inversores de potência, alcançando redução de recursos de *hardware* em comparação com soluções comerciais.

De forma semelhante, Aspar et al. (2025) focaram no controle de uma máquina *HY table* utilizando um escravo *Modbus* em *FPGA* para acionar motores de passo, validando a confiabilidade do sistema em bancada com latência na ordem de nanossegundos. Ambos os trabalhos são limitados a um único escravo e a um conjunto restrito de funções *Modbus*, não abordando a comunicação com múltiplos dispositivos em uma mesma rede. O presente trabalho diferencia-se ao propor uma arquitetura de rede completa com mestre e múltiplos escravos digitais simulados internamente, superando essa limitação através da multiplexação de escravos em um barramento serial interno e da implementação de uma máquina de estados capaz de gerenciar transações concorrentes.

No campo de sistemas de controle e gateway, Kurniawan et al. (2024) propuseram uma arquitetura *PWM* multicanal integrada ao *Modbus RTU*, com suporte às funções 0x03, 0x04, 0x06 e 0x10, mas limitaram-se a um único escravo com quatro canais de saída, sem simular uma rede com múltiplos dispositivos. Por sua vez, Wibowo et al. (2026) desenvolveram um *System-on-Chip* em *FPGA* para consolidar dados de oito sensores via mestre *Modbus RTU* e transmiti-los por LoRa, alcançando temporização determinística, porém com arquitetura voltada exclusivamente para coleta de dados.

Zhang et al. (2026) projetaram um *gateway* para conversão bidirecional entre *Modbus TCP* e *RTU*, superando limitações de *gateways* baseados em

microcontroladores, mas com foco na interoperabilidade entre redes, não na simulação de uma rede industrial completa. Em contraste, este trabalho não apenas implementa um escravo, mas também um mestre simulado e três escravos com bancos de registradores independentes, permitindo a validação do comportamento completo da rede em um único dispositivo *FPGA*, com ênfase na modularidade e na correta implementação das restrições temporais do protocolo.

A principal lacuna identificada nos trabalhos relacionados é a ausência de uma arquitetura que integre, no mesmo ambiente de simulação, todos os elementos de uma rede *Modbus RTU* (mestre, barramento e múltiplos escravos) permitindo a validação funcional completa sem dependência de *hardware* externo. Enquanto as pesquisas existentes concentram-se em aplicações específicas (controle de motores, *PWM*, *gateway* ou aquisição de dados), o presente projeto contribui com uma solução didática e escalável que simula sensores e atuadores por meio de bancos de registradores internos, implementa as funções 03 e 06 do protocolo e incorpora um módulo de CRC-16 para detecção de erros.

Dessa forma, o trabalho oferece uma base reutilizável para futuras integrações com sensores físicos, atuadores reais ou interfaces RS-485, preenchendo a lacuna de validação de redes *Modbus* em *FPGA* em nível de sistema. A Tabela 1 resume, de forma comparativa, os trabalhos relacionados e o presente projeto.

Tabela 1 – Comparação entre os trabalhos relacionados e este projeto

| Autor e Ano         | Objetivo Principal                                                     | Método Utilizado                                         | O que falta / Lacuna                                  |
|---------------------|------------------------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------|
| McBride (2023)      | Controlar inversores de potência com <i>Modbus RTU</i> em <i>FPGA</i>  | <i>IP cores</i> em <i>VHDL</i> , escravo único           | Não aborda rede com múltiplos escravos                |
| Aspar et al. (2025) | Controlar máquina <i>HY table</i> com <i>Modbus RTU</i> em <i>FPGA</i> | <i>Verilog HDL</i> , escravo único para motores de passo | Limitado a funções específicas; sem simulação de rede |

|                         |                                                                               |                                                                                               |                                                                                                        |
|-------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Kurniawan et al. (2024) | Gerar <i>PWM</i> multicanal com <i>Modbus RTU</i> em <i>FPGA</i>              | Arquitetura com <i>UART</i> e registradores, escravo único                                    | Sem múltiplos escravos; foco apenas em <i>PWM</i>                                                      |
| Wibowo et al. (2026)    | Aquisição de 8 sensores via <i>Modbus RTU</i> e transmissão LoRa              | SoC com mestre <i>Modbus</i> e CRC16                                                          | Arquitetura dedicada a coleta, sem simular rede de escravos                                            |
| Zhang et al. (2026)     | Gateway de conversão <i>Modbus TCP/RTU</i> em <i>FPGA</i>                     | <i>Core soft TCP/IP</i> + FIFOs assíncronos                                                   | Foco em interoperabilidade, não em simulação de rede industrial<br>não em simulação de rede industrial |
| Este Trabalho           | Simular rede <i>Modbus RTU</i> com mestre e múltiplos escravos em <i>FPGA</i> | <i>SystemVerilog</i> , <i>FSM</i> , bancos de registradores internos, funções 03 e 06, CRC-16 | (preencher a lacuna com simulação completa de rede)                                                    |

## 4 METODOLOGIA DE PESQUISA

Com relação à natureza desta pesquisa, é caracterizada como aplicada, adotando uma abordagem predominantemente experimental com suporte de revisão bibliográfica e documental (PRODANOV; FREITAS, 2013). O desenvolvimento do projeto é baseado em uma estratégia incremental, em que as definições de parâmetros, interfaces e testes precedem a implementação e a validação isolada de cada bloco, o que irá resultar posteriormente na integração do sistema completo.

A metodologia desta pesquisa propõe que a arquitetura da rede simulada seja modularizada em três camadas principais: camada de transporte serial, processamento do protocolo Modbus e o nível superior de integração, o *top-level*. Com o objetivo de manter o transporte independente do protocolo, a camada de transporte serial foca no fluxo de temporização e na tradução de *bits*, sendo constituída pelos módulos conversores UART, pelo gerador de *baud rate* e por um temporizador para detectar intervalos de silêncio e delimitar os quadros.

A camada do protocolo Modbus concentra a conformidade do pacote com as regras da Modbus Organization. Nesta camada, atuam o módulo responsável pela extração e decodificação dos campos, e o módulo de cálculo e verificação iterativa da assinatura de erro CRC-16. O controle operacional é gerido por Máquinas de Estados Finitos (FSM), que orquestram a rotina de envio, a espera e a resposta às funções de leitura 03 (*Read Holding Registers*) e de escrita 06 (*Write Single Register*). Para espelhar fisicamente sensores e atuadores da rede, incorpora-se o módulo de banco de registradores internos (MODBUS ORGANIZATION, 2006).

Por fim, a camada de integração (*top-level*) realiza a conexão de todas as entidades projetadas, modelando o meio compartilhado interno e unindo o mestre às instâncias paramétricas de escravos Modbus. Essa organização assegura que as respostas ocorram exclusivamente quando o endereço do escravo no quadro bater com seu identificador na rede. Além disso, cada submódulo recebe testes unitários autônomos com o objetivo de assegurar a aderência entre a arquitetura e os

requisitos normativos do protocolo. Também são empregados *testbenches*, que geram requisições de mestre, aguardam a resposta da rede e conferem automaticamente o status final comparando com as respostas esperadas. O plano de testes também engloba a execução de operações bem-sucedidas e também de injeção de falhas e tratamentos de exceção, validando o comportamento da FSM perante erros de cálculo no CRC, requisições a registradores ou IDs inexistentes, limites de tempo estourados (*timeouts*), uso de função não suportada e eventuais colisões de tráfego interno.

Para a construção da topologia, optou-se por uma arquitetura em que o mestre e múltiplos escravos internos compartilham o mesmo sinal de *clock* e um barramento serial interno. Essa abordagem foi escolhida por demonstrar claramente o comportamento de uma rede com múltiplos nós concorrentes, mesmo na ausência inicial de componentes físicos externos. Além disso, a metodologia inclui a modelagem em simulação do canal correspondente ao padrão RS-485, incorporando efeitos práticos como contenção, tempo de *turnaround*, atrasos e inserção de ruído, com o intuito de avaliar a robustez das FSMs.

O código SystemVerilog validado é destinado ao fluxo de síntese lógica. Essa atividade permite extrair dados importantes para sistemas embarcados, avaliando quantitativamente a utilização de recursos estruturais, contabilizando as tabelas de busca (LUTs), componentes de memória, registradores (*Flip-Flops*), etc., por exemplo, e o alcance da frequência máxima de *clock* na placa FPGA selecionada - a decidir. Esse mapeamento fundamentará a discussão dos resultados de modo a evidenciar os limites de *hardware*, o custo lógico da FPGA por escravo e a escalabilidade inerente à rede industrial em níveis eletrônicos.

#### 4.1 Materiais e Métodos

Como referência normativa para modelar o comportamento físico e lógico do sistema, foram utilizadas as especificações oficiais Modbus Application Protocol V1.1b3 e Modbus over Serial Line V1.02 (MODBUS ORGANIZATION, 2006, 2012). Para a descrição dos circuitos em nível RTL e para a criação dos *testbenches*, é utilizada a linguagem *SystemVerilog*, em conformidade com a padronização IEEE 1800-2023 (IEEE, 2024). O fluxo de desenvolvimento utiliza dois ambientes

principais de *software*, responsáveis pelas etapas de simulação funcional e de síntese para o *hardware*, são eles o Intel Quartus Prime, que atua como compilador dos módulos e como ferramenta de síntese para a tradução e o mapeamento dos blocos lógicos na arquitetura física do dispositivo; e o QuestaSim, utilizado principalmente para execução dos testes unitários e a simulação do *testbench*. Além disso, foi utilizado um sistema de controle de versão, o Git, para gerenciar e documentar a integração dos módulos, e o Github para hospedagem.

#### 4.1.1 Módulo de Transporte Serial e Controle de Temporização

Devido às rigorosas restrições temporais exigidas pelo protocolo, a arquitetura implementada delega o controle de temporização (*timing*) ao módulo de transporte serial e a um temporizador dedicado (*frame\_timer*). Este temporizador atua de forma autônoma para monitorar os intervalos de silêncio e é o responsável por informar à unidade mestre o instante exato em que a recepção de um quadro da rede foi concluída.

Para o processamento lógico, a Máquina de Estados Finitos (FSM) foi projetada para operar exclusivamente por meio de uma interface orientada a *bytes*. O módulo UART desempenha o papel de *rx\_data*, *rx\_valid* e *rx\_frame\_end*, enquanto a transmissão é comandada através das vias *tx\_data*, *tx\_valid* e *tx\_ready*, conforme Apêndice A. Essa separação arquitetural em camadas evita o acoplamento indevido entre as regras normativas do protocolo e os processos de amostragem do sinal serial de baixo nível, conferindo alta modularidade ao sistema e facilitando a execução de testes unitários isolados.

#### 4.1.1 Módulo Mestre Modbus RTU

O módulo mestre Modbus RTU foi desenvolvido com o objetivo de controlar a emissão de requisições, a recepção de respostas e a validação das transações associadas às funções 03 (*Read Holding Registers*) e 06 (*Write Single Register*). Sua implementação foi realizada em *SystemVerilog*, em nível RTL, adotando uma arquitetura modular composta por pacotes de definição, pacote de CRC, montador de quadros, máquina de estados finitos e testbench de verificação funcional.

A implementação do mestre foi organizada em arquivos distintos. O pacote *modbus\_defs\_pkg.sv* concentra os códigos das funções *Modbus* suportadas, o limite

máximo de leitura de registradores e o tipo enumerado *modbus\_status\_t*, utilizado para representar os resultados da transação. O pacote *modbus\_crc\_pkg.sv* contém a função compartilhada de cálculo incremental do CRC-16 *Modbus*, evitando duplicação de lógica entre o montador de quadros, a máquina de estados e o testbench. O módulo *master\_frame\_builder.sv* é responsável pela montagem da requisição *Modbus* RTU, organizando os *bytes* de endereço do escravo, código de função, endereço do registrador, dado ou quantidade e campo CRC. O controle da comunicação é realizado pelo módulo *modbus\_master\_fsm.sv*, que coordena a sequência de operação do mestre.

A máquina de estados do mestre foi estruturada em etapas correspondentes à aceitação do comando, montagem da requisição, transmissão dos *bytes*, espera da resposta, validação do quadro recebido e finalização da transação. A interface de comando recebe o endereço do escravo, o código de função, o endereço do registrador e o dado ou quantidade associada à operação. A transmissão utiliza os sinais *tx\_data*, *tx\_valid* e *tx\_ready*, enquanto a recepção utiliza *rx\_data*, *rx\_valid* e *rx\_frame\_end*. Essa separação permite que o módulo mestre permaneça independente da implementação física da UART, operando apenas sobre uma interface orientada a bytes.

Para a função 03, o mestre valida a quantidade de *bytes* recebidos, o CRC, o endereço do escravo, o código de função e a coerência entre a quantidade solicitada e a quantidade retornada. Os registradores lidos são disponibilizados por meio de *response\_registers*, *response\_register\_count* e também por uma interface de leitura por índice, composta por *response\_register\_read\_index*, *response\_register\_read\_data* e *response\_register\_read\_valid*. Para a função 06, o mestre verifica se a resposta ecoa corretamente o endereço do registrador e o valor escrito, conforme previsto pela especificação *Modbus*.

Também foram implementados mecanismos de robustez para tratar condições anormais de comunicação. Entre essas condições estão CRC inválido, resposta de exceção, endereço inesperado, função incompatível, quadro incompleto, *timeout*, *overflow* de resposta, função não suportada e parâmetros inválidos. No caso da função 03, comandos com quantidade igual a zero, superior a 125 registradores ou

acima do limite configurado por *MAX\_READ\_REGISTERS* são rejeitados antes da transmissão, retornando *STATUS\_INVALID\_PARAM*.

A verificação funcional do módulo mestre foi realizada por meio do testbench *tb\_modbus\_master\_fsm.sv*. Esse *testbench* aplica comandos ao mestre, simula respostas de escravos e compara automaticamente os resultados obtidos com os estados esperados. Foram contemplados cenários de leitura válida, escrita válida, leitura de múltiplos registradores, leitura por índice, resposta de exceção, CRC inválido, endereço incorreto, função incorreta, quadro curto, *byte count* inválido, *overflow*, função não suportada, parâmetros inválidos, *timeout* por ausência de resposta e *timeout* após recepção parcial.

Os arquivos foram compilados com o compilador do *ModelSim*, seguindo a ordem: *modbus\_defs\_pkg.sv*, *modbus\_crc\_pkg.sv*, *master\_frame\_builder.sv*, *modbus\_master\_fsm.sv* e *tb\_modbus\_master\_fsm.sv*. A compilação foi concluída sem erros e sem avisos, evidenciando a consistência sintática e estrutural dos módulos *SystemVerilog* desenvolvidos. **A execução completa da simulação com *ModelSim* depende de uma licença funcional do ambiente Questa/Intel que será verificado a melhor forma para substituição e validação.**

#### 4.1.1 Módulo Escravo

##### 4.1.1 Módulo CRC-16

O módulo de verificação de erro *modbus\_crc16.sv* (**Apêndice X**) foi projetado para cumprir rigorosamente o algoritmo matemático estipulado pela norma oficial do protocolo, empregando o polinômio gerador de reflexão *0xA001* e a pré-carga do registrador de vetor em *0xFFFF*. No entanto, visando explorar o paralelismo e otimizar o desempenho temporal da rede, a arquitetura proposta substituiu o tradicional laço de repetição sequencial por blocos de lógica estritamente combinacional (*always\_comb*). Através do desenrolamento (*loop unrolling*) das oito iterações de deslocamento (*shift*) e operações de *XOR*, o hardware desenvolvido consolida o cálculo do CRC de um byte completo em um único ciclo de *clock*, desonerando o sistema de sobrecargas de processamento e latências acumuladas. O registrador de estado do CRC é atualizado de forma sequencial síncrona apenas



mediante a validação de um pulso de habilitação (*enable*) comandado pelo decodificador do escravo, contando ainda com um sinal de limpeza (*clear*) síncrono para reinicializar a semente em cada início de mensagem.

#### 4.1.2 Módulo de Registradores

O armazenamento interno de dados foi materializado no módulo *modbus\_register\_file.sv* (Apêndice Y), cuja função primária é emular o mapa de memória de sensores e atuadores industriais do escravo digital através de registradores do tipo *Holding Registers*. O circuito foi estruturado como uma matriz de memória de 16 bits com profundidade parametrizável, configurada inicialmente para 64 posições. A operação de escrita no banco ocorre de maneira síncrona na borda de subida do sinal de *clock* (*always\_ff*), condicionada à ativação do pino de habilitação de escrita (*write enable*) acionado após a validação da Função Modbus 06. Para mitigar falhas de violação de escopo, incorporou-se uma lógica de proteção que valida se o endereço solicitado pelo mestre encontra-se dentro dos limites físicos da profundidade da memória. Em contrapartida, a leitura de dados para o atendimento à Função Modbus 03 foi mapeada de forma puramente combinacional (*always\_comb*), garantindo que o dado contido no registrador alvo seja disponibilizado instantaneamente para o subsistema de montagem do quadro de resposta do escravo.

## **5      RESULTADOS E AVALIAÇÃO**

## **6 CONCLUSÃO**

## **Apêndice A - Módulo de Transporte Serial e Controle de Temporização**

## **Apêndice B - Módulo Mestre**

## **Apêndice C - Módulo Escravo**

## **Apêndice X - Módulo CRC-16**

**Descrição:** módulo que irá receber os bytes do decodificador e atualizar o cálculo do CRC.

- Código do módulo *modbus\_crc16.sv*

|  |
|--|
|  |
|--|

- Código de teste do módulo *tb\_modbus\_crc16.sv*

|  |
|--|
|  |
|--|

## **Apêndice Y - Módulo de Registradores**

**Descrição:** memória interna que simula os sensores e atuadores.

- Código do módulo *modbus\_register\_file.sv*

- Código de teste do módulo *tb\_modbus\_register\_file.sv*

## REFERÊNCIAS

AMD. Vivado Design Suite User Guide: Synthesis (UG901), FSM Description. Version 2022.2. San Jose: AMD, 2022. Disponível em: <https://docs.amd.com/r/2022.2-English/ug901-vivado-synthesis/FSM-Description>.

Acesso em: 2 jul. 2026.

AMD. Vivado Design Suite Properties Reference Guide (UG912), FSM\_SAFE\_STATE. Version 2023.2. San Jose: AMD, 2023. Disponível em: [https://docs.amd.com/r/2023.2-English/ug912-vivado-properties/FSM\\_SAFE\\_STATE](https://docs.amd.com/r/2023.2-English/ug912-vivado-properties/FSM_SAFE_STATE).

Acesso em: 2 jul. 2026.

ASPAR, Z.; ABD RAHMAN, N. H.; ASPAR, M. H. Modbus RTU on FPGA to Control HY Table Machine via a PC. *In: IEEE INTERNATIONAL CONFERENCE ON ELECTRICAL, CONTROL AND COMPUTER ENGINEERING (InECCE)*, 2025. **Anais [...]** [S. l.]: IEEE, 2025.

MCBRIDE, B. **Modbus RTU for Embedded Cyber Secure Inverter Controller**. 2023. Dissertação (Mestrado) – University of Arkansas, Fayetteville, 2023.

ZHANG, C. *et al.* Design of Modbus TCP/RTU Communication Gateway Based on FPGA. **IEEE Xplore**, [S. l.], 2026.

WIBOWO, F. W. *et al.* System-on-Chip of 8-in-1 Sensor Data Transmission to LoRa Communication Based on Field Programmable Gate Array. **IEEE Xplore**, [S. l.], 2026.

KURNIAWAN, S. *et al.* **Design and Simulation Multichannel PWM Based on Modbus Protocol in FPGA Device**. Jambi: Politeknik Jambi, 2024.

DARJI, A. D. FPGA-Based Display Driver Design with Remote Terminal Unit (RTU) Support Using MODBUS. *In: Recent Trends in Electronics and Communication*. [S.l.]: Springer, 2022. v. 777, p. 511.

CHU, Pong P. FPGA Prototyping by SystemVerilog Examples. John Wiley & Sons, 2011.

HU, Sideng; ZHAO, Zhengming; ZHANG, Yingchao; WANG, Shuping. A novel Modbus RTU-based communication system for adjustable speed drives. 2008 IEEE Vehicle Power and Propulsion Conference, Harbin, p. 1-5, 2008. DOI: 10.1109/VPPC.2008.4677584. Disponível em: <https://ieeexplore.ieee.org/document/4677584>. Acesso em: 01 jul. 2026.

IEEE. IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. 28 fev. 2024.

JOHNSON, E. FPGA-Based Embedded Systems for Industrial Automation. American Journal of Embedded Systems and VLSI Design, v. 1, n. 5, p. 8-12, 2020.

MODBUS ORGANIZATION. MODBUS Application Protocol Specification V1.1b3. 2012. Disponível em: <https://www.modbus.org/>. Acesso em: 25 jun. 2026.

MODBUS ORGANIZATION. MODBUS over Serial Line Specification and Implementation Guide V1.02. 2006. Disponível em: <https://www.modbus.org/>. Acesso em: 25 jun. 2026.

NIST. Guide to Operational Technology (OT) Security: NIST Special Publication 800-82 Revision 3. Gaithersburg: National Institute of Standards and Technology, 2023. Disponível em: <https://csrc.nist.gov/pubs/sp/800/82/r3/final>. Acesso em: 25 jun. 2026.

PAGARE, R. Implementation of Modbus on FPGA. SSRN Electronic Journal, 28 mar. 2020.

PRODANOV, C. C.; FREITAS, E. C. de. **Metodologia do Trabalho Científico: Métodos e Técnicas da Pesquisa e do Trabalho Acadêmico**. 2. ed. Editora Feevale, 2013.

SIQUEIRA, R. Estudo e Implementação do Protocolo de Comunicação Modbus RTU no Servoconversor SCA700. 2024. Trabalho de Conclusão de Curso (Graduação) – Universidade Federal de Santa Catarina, Florianópolis, 2024.

SZÁSZ, Csaba. Cyber-physical platform development and implementation for industry 4.0. *International Review of Applied Sciences and Engineering*, p. 1-7, 2020. DOI: 10.1556/1848.2020.00010. Disponível em: [https://www.researchgate.net/publication/340889822\\_Cyber-physical\\_platform\\_development\\_and\\_implementation\\_for\\_industry\\_40](https://www.researchgate.net/publication/340889822_Cyber-physical_platform_development_and_implementation_for_industry_40). Acesso em: 01 jul. 2026.

CHHATRAWALA, Jaimeen; JASANI, Nandish Bharatbhai; TILVA, Vidita. FPGA based data acquisition with Modbus protocol. In: *INTERNATIONAL CONFERENCE ON COMMUNICATION AND SIGNAL PROCESSING*, 2016. Proceedings [...]. IEEE, 2016. p. 1251-1254. DOI: 10.1109/ICCSP.2016.7754353.

IEEE. IEEE Std 1800-2023: IEEE Standard for SystemVerilog - Unified Hardware Design, Specification, and Verification Language. New York: IEEE, 2024.

MODBUS ORGANIZATION. MODBUS Application Protocol Specification V1.1b3. 2012. Disponível em: [https://www.modbus.org/docs/Modbus\\_Application\\_Protocol\\_V1\\_1b3.pdf](https://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf). Acesso em: 2 jul. 2026.

MODBUS ORGANIZATION. MODBUS over Serial Line Specification and Implementation Guide V1.02. 2006. Disponível em: [https://www.modbus.org/docs/Modbus\\_over\\_serial\\_line\\_V1\\_02.pdf](https://www.modbus.org/docs/Modbus_over_serial_line_V1_02.pdf). Acesso em: 2 jul. 2026.

OLMEDO, Sergio; GUANUCO, Luis; PANOZZO ZENERE, Jonatan; RUBIO, Agustin. Implementación de MODBUS en FPGA mediante VHDL: capa de enlace. In: *CONGRESO DE MICROELECTRÓNICA APLICADA*, 2., 2011, La Plata. Libro de Memorias [...]. La Plata, 2011. p. 101-105. Disponível em: <http://sedici.unlp.edu.ar/handle/10915/121538>. Acesso em: 2 jul. 2026.



PAGARE, Prajakta et al. Implementation of Modbus on FPGA. In: INTERNATIONAL CONFERENCE ON INNOVATIVE COMPUTING & COMMUNICATIONS, 2020. Proceedings [...]. 2020. DOI: 10.2139/ssrn.3562960.

STOUFFER, Keith et al. Guide to Operational Technology (OT) Security. Gaithersburg: National Institute of Standards and Technology, 2023. NIST SP 800-82 Rev. 3. DOI: 10.6028/NIST.SP.800-82r3.

URBINA, Marcelo; LÁZARO, Jesús; ASTARLOA, Armando; ACOSTA, T. E. CPPS Gateway: implementation of Modbus and Profibus on a SoC programmable platform. IEEE Latin America Transactions, v. 16, n. 2, p. 335-341, 2018. DOI: 10.1109/TLA.2018.8327384.

URREA, Claudio; MORALES, Claudio; KERN, John. Implementation of error detection and correction in the Modbus-RTU serial protocol. International Journal of Critical Infrastructure Protection, v. 15, p. 27-37, 2016. DOI: 10.1016/j.ijcip.2016.07.001.